Migrating Security-Sensitive Proxy Components

from C++ to Memory-Safe Rust

A simple-language guide to the project: summary, components, workflow and architecture

> **The project in one sentence (as written)**
> Migrated security-sensitive parsing and policy components to memory-safe Rust without rewriting the existing C++ proxy stack, using ownership and borrowing, traits, Result-based error handling, async execution, bounded concurrency, and versioned FFI boundaries for incremental rollout at unchanged data-plane performance.

Contents

1. Overall summary (in simple terms)
2. Decoding the project sentence, phrase by phrase
3. Key ideas explained simply (a mini-glossary)
4. Architecture diagram and a full walk-through of it
5. Components explained in detail
6. Workflows explained end to end
7. How performance stayed unchanged
8. Security benefits and what the project achieved
9. Quick recap

Note: the project sentence is a short summary. This guide describes a realistic, typical design for such a project so that every part of the sentence can be explained concretely. Component names (for example “Rust hook shim” or “Policy snapshot store”) are descriptive labels chosen for this guide.

1. Overall Summary (in Simple Terms)

Imagine a very busy **security gate** at the entrance of a big office building. Every visitor (a network request) passes through this gate. The gate checks who the visitor is, reads their papers, and decides whether to let them in and which floor to send them to. In computing, this gate is called a **proxy**. This particular gate was built many years ago in the **C++** programming language. It is fast and reliable, but C++ has one well-known weakness: it lets programmers make **memory mistakes** (reading or writing the wrong piece of computer memory). Attackers love these mistakes, because a carefully crafted bad request can exploit them to crash the gate or even take control of it.

The riskiest jobs at the gate are the two that deal directly with untrusted input from strangers:

- **Parsing** – reading and understanding the raw bytes a visitor sends (headers, tokens, URLs, cookies). This is like reading a visitor's handwritten papers, which might be deliberately confusing or malformed.
- **Policy decisions** – applying security rules ("Is this user allowed?", "Is this path blocked?", "Is the rate too high?").

This project moved **only those risky jobs** into **Rust**, a modern language that prevents memory mistakes automatically at compile time (before the program ever runs). Everything else in the gate stayed in C++, untouched. It is like replacing only the lock and the ID-scanner of a building with a much stronger model, instead of tearing down and rebuilding the whole building.

To make the old C++ code and the new Rust code work together safely, the team built a carefully designed **connection point** between them, called an **FFI boundary** (Foreign Function Interface). This connection point has a **version number**, so both sides always agree on exactly how they talk to each other, and so new versions can be introduced gradually.

The switch was done **step by step** (incremental rollout): first the Rust code ran silently next to the C++ code just to compare answers ("shadow mode"), then it handled a small percentage of real traffic, then more, and finally all of it, while the old C++ code stayed available as an instant fallback. Throughout this process, the gate remained **just as fast as before** (unchanged data-plane performance), which matters because every millisecond of delay is felt by every user.

> **In one breath**
>
> Old, fast C++ gate kept as it is → the dangerous "reading" and "rule-checking" parts rewritten in safe Rust → joined through a versioned, carefully controlled connection → switched on gradually with a safety net → safer gate, same speed.

2. Decoding the Project Sentence, Phrase by Phrase

Each phrase of the project sentence carries a specific meaning. The table below translates every phrase into plain language.

Phrase	What it means in simple words
Migrated	Moved existing functionality from one language (C++) to another (Rust). The behaviour stays the same; the implementation changes.
Security-sensitive parsing components	Code that reads raw, untrusted bytes from the network (HTTP headers, URLs, cookies, tokens such as JWTs) and turns them into structured data. Bugs here are the most common source of security holes.
Policy components	Code that applies security rules to a request: access control, allow/deny lists, rate limits, header rewriting rules. A wrong answer here means a wrong security decision.
Memory-safe Rust	Rust checks at compile time that the program never uses memory after freeing it, never reads past the end of a buffer, and never has two threads changing the same data at once. Whole classes of bugs simply cannot be written.
Without rewriting the existing C++ proxy stack	The rest of the proxy (networking, TLS, routing, load balancing) was left alone. Only small, high-risk pieces were replaced. This keeps risk, cost and time low.
Ownership and borrowing	Rust's core rules for memory: every value has exactly one owner who is responsible for freeing it; others can borrow it temporarily without copying. This is how parsing avoids both bugs and expensive copies.
Traits	Rust's way of describing a shared set of abilities (like an interface). Every policy rule implements the same trait, so new rules can be plugged in without changing the engine.
Result-based error handling	Functions return either `Ok(value)` or `Err(error)`. The compiler forces the caller to handle the error case. No hidden crashes, no ignored failures.
Async execution	Slow background jobs (like downloading new policy files) run without blocking the fast request path, using an asynchronous runtime (Tokio).
Bounded concurrency	A hard limit on how many background jobs can run at once (for example with a semaphore). This stops overload and protects CPU and memory under stress.

Phrase	What it means in simple words
Versioned FFI boundaries	The doorway between C++ and Rust is a plain C-style interface that carries a version number. Both sides check the version at start-up, so mismatched builds are rejected safely and new versions can coexist with old ones.
Incremental rollout	Turning the new code on gradually (shadow → 1% → 10% → 50% → 100%) with the ability to switch back instantly.
Unchanged data-plane performance	The “data plane” is the hot path every request travels. Latency (delay), throughput (requests per second) and CPU/memory use stayed the same as before the migration.

3. Key Ideas Explained Simply (a Mini-Glossary)

Before looking at the architecture, it helps to understand the building blocks. Each idea is explained with an everyday analogy.

3.1 Proxy, data plane and control plane

- **Proxy:** a middleman server that sits between users and the real applications. It receives every request, checks it, and forwards it. Think of a receptionist who screens and redirects every visitor.
- **Data plane:** the part that handles live traffic, request by request. It must be extremely fast. This is the receptionist actually dealing with visitors.
- **Control plane:** the part that decides the rules and sends them to the data plane. This is the manager who updates the receptionist's rule book.

3.2 Memory safety

Computer memory is like a long row of numbered lockers. C++ lets a program open any locker number, even one that belongs to someone else or that was already emptied. Typical mistakes are:

- **Buffer overflow:** writing past the end of your lockers into the neighbour's.
- **Use-after-free:** using a locker after you handed back the key; someone else may now be using it.
- **Double free:** handing back the same key twice, confusing the locker manager.
- **Data race:** two people changing the same locker's contents at the same time.

Rust's compiler acts like a strict locker manager who checks every action before the program is allowed to run, so these mistakes are rejected up front.

3.3 Ownership and borrowing

- **Ownership:** every piece of data has exactly one owner. When the owner goes away, the data is cleaned up automatically. Like a library book that is always checked out to exactly one person.
- **Borrowing:** others can look at the data without taking it. Many people may read it at once (shared borrow, `&T`), or exactly one person may edit it (mutable borrow, `&mut T`), never both at the same time.
- **Lifetimes:** the compiler checks that nobody keeps a borrowed reference longer than the owner keeps the data. A borrowed photocopy cannot outlive the original.
- **Why it matters here:** the parser can point directly into the request bytes (zero-copy) instead of copying them, which is both safe and fast.

3.4 Traits

A trait is a promise of abilities. If a type implements the trait `Rule`, it promises it can `evaluate(request)` and return a decision. The policy engine does not care whether the rule is an IP block list or a JWT check; it only needs the promise. Like a power socket: any appliance with the right plug works.

3.5 Result-based error handling

In Rust, a function that can fail returns `Result<T, E>`, which is either `Ok(T)` (success, here is the value) or `Err(E)` (failure, here is why). The compiler will not let you use the value without first dealing with the possibility of failure. It is like a parcel that can only be opened after you sign that you checked whether it arrived damaged.

3.6 Async execution and bounded concurrency

- **Async:** a way to start a slow task (such as a network download) and do other useful work while waiting, instead of standing idle. Like putting a kettle on and making toast while the water heats.
- **Tokio:** the most widely used async runtime for Rust; it schedules thousands of small tasks onto a few threads.
- **Bounded concurrency:** a cap on how many tasks may run at once, usually with a **semaphore** (a counter of available "permits"). Like a restaurant with a fixed number of tables; extra guests wait instead of overcrowding the kitchen.

3.7 FFI, ABI and versioning

- **FFI (Foreign Function Interface):** the mechanism that lets code in one language call functions written in another. Here, C++ calls Rust.
- **ABI (Application Binary Interface):** the exact low-level agreement about how data is laid out and passed in a function call. Both sides must agree byte for byte.
- **C ABI:** the simplest, most stable common language that both C++ and Rust understand. Rust exposes functions marked `extern "C"`.
- **Versioned boundary:** the interface carries a version number (for example v1, v2). The C++ side asks "which version do you speak?" at start-up and refuses to use a mismatched library. Like two diplomats confirming they are using the same edition of a treaty.

4. Architecture Diagram and Full Walk-Through

The diagram on the next page shows the whole system. It has three vertical zones and four external systems. Numbered circles on the arrows show the order in which things happen. The walk-through after the diagram explains every element.

How to read the diagram	Meaning
Blue zone (left)	The existing C++ proxy. Its boxes are numbered 1 to 7. Nothing here was rewritten except a small hook (box 4).
Purple zone (middle)	The FFI boundary: the controlled doorway between C++ and Rust. Its five small boxes are the rules of that doorway.
Orange zone (right)	The new Rust components, lettered A to G.
Green boxes (top and bottom)	External systems: clients, control plane, backend services, observability stack.
Dark blue arrows 1–10	The live request path that every request follows.
Grey dashed arrows F	The fallback path to the legacy C++ parser, used when Rust is switched off or unavailable.
Brown arrows P1–P4	The policy update path (runs in the background, not per request).
Green arrows T1–T2	The telemetry path (metrics and comparison results).

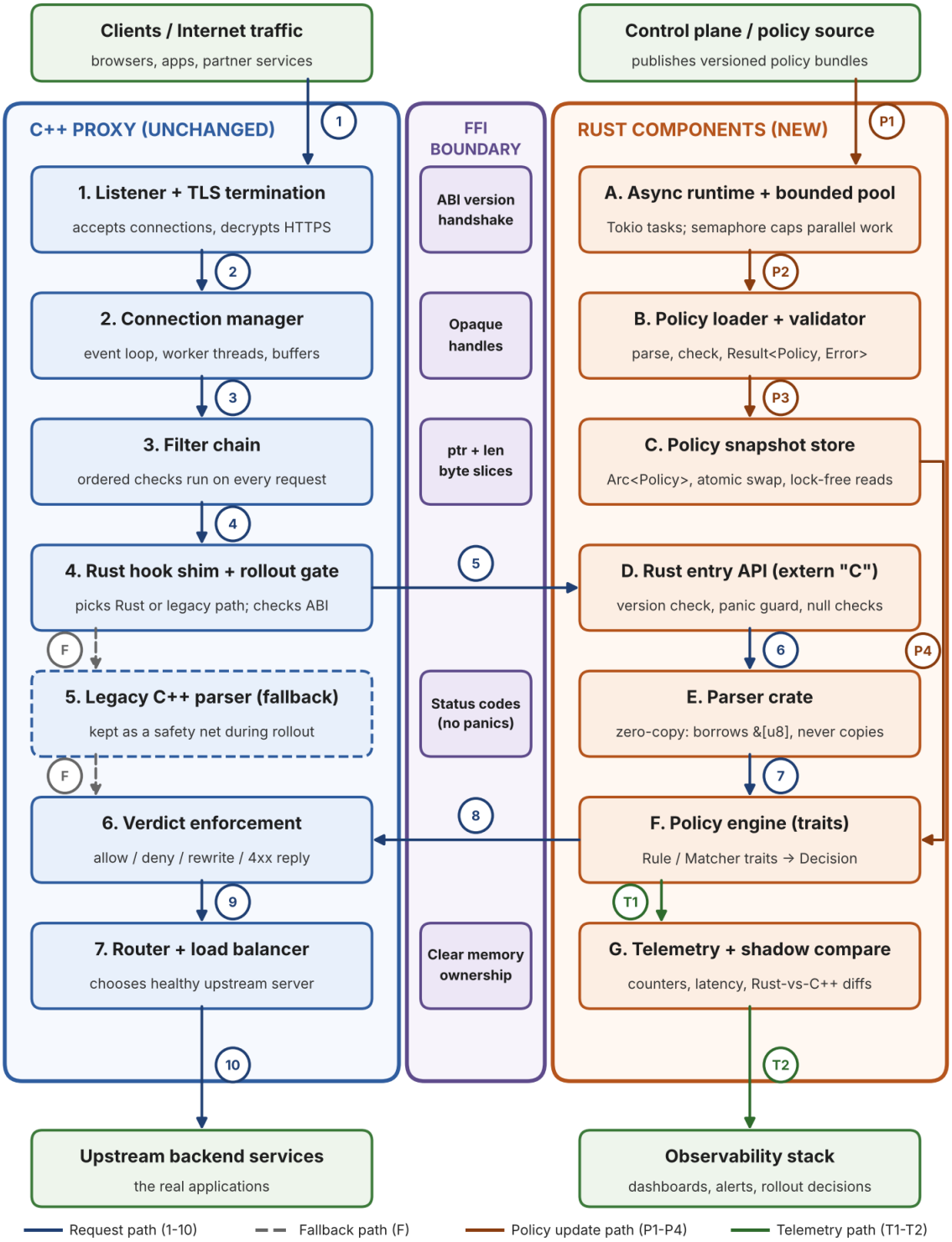


Figure 1. System architecture: existing C++ proxy, versioned FFI boundary and new Rust components

4.1 The three zones

Blue zone – C++ proxy (unchanged). This is the original proxy that already handled production traffic. It keeps doing everything it was good at: accepting connections, TLS encryption, managing worker threads and buffers, routing and load balancing. The only new code inside it is a thin hook (box 4) that can hand work to Rust.

Purple zone – FFI boundary. This is not a separate program; it is the agreed contract between the two languages. It is drawn as its own zone because it is the most carefully designed part of the project. Every piece of data that crosses from left to right or right to left must follow its five rules.

Orange zone – Rust components (new). All new, memory-safe code lives here. It is compiled into a library that is loaded into the same process as the C++ proxy, so calls are ordinary function calls (nanoseconds), not network calls.

4.2 Every box in the diagram

Box	Zone	One-line explanation
Clients / Internet traffic	External	Anyone sending requests: browsers, mobile apps, partner systems. Their input is untrusted.
Control plane / policy source	External	The system that publishes security rule bundles, each with a version number.
1. Listener + TLS termination	C++	Accepts network connections and decrypts HTTPS into plain bytes.
2. Connection manager	C++	Runs the event loop and worker threads, owns the memory buffers that hold request bytes.
3. Filter chain	C++	An ordered list of checks every request passes through.
4. Rust hook shim + rollout gate	C++ (new, small)	A thin C++ wrapper that decides whether this request goes to Rust or to the legacy parser, and calls Rust through the FFI.
5. Legacy C++ parser (fallback)	C++ (dashed)	The old parsing/policy code, kept as a safety net until rollout finished.
6. Verdict enforcement	C++	Applies the final decision: forward, block, rewrite headers or reply with an error.
7. Router + load balancer	C++	Chooses which healthy backend server gets the request.
ABI version handshake	FFI rule	Both sides confirm the interface version at start-up.

Box	Zone	One-line explanation
Opaque handles	FFI rule	C++ holds Rust objects only as anonymous pointers it cannot look inside.
ptr + len byte slices	FFI rule	Data is passed as a pointer plus a length, so Rust knows exactly where the bytes end.
Status codes (no panics)	FFI rule	Errors cross as simple numbers; Rust crashes (panics) are never allowed to cross.
Clear memory ownership	FFI rule	For every piece of memory it is written down which side allocates and which side frees it.
A. Async runtime + bounded pool	Rust	Runs background work on Tokio, with a semaphore limiting how many jobs run at once.
B. Policy loader + validator	Rust	Downloads, parses and checks new policy bundles; rejects bad ones safely.
C. Policy snapshot store	Rust	Holds the current rule set; swaps in new versions atomically without locks.
D. Rust entry API (extern "C")	Rust	The functions C++ actually calls; checks inputs and catches any panic.
E. Parser crate	Rust	Reads raw request bytes into structured data without copying them.
F. Policy engine (traits)	Rust	Runs every rule (each implements a common trait) and produces a decision.
G. Telemetry + shadow compare	Rust	Records metrics and compares Rust's answer with the C++ answer during rollout.
Upstream backend services	External	The real applications that finally receive allowed requests.
Observability stack	External	Dashboards and alerts used by engineers to decide when to widen the rollout.

4.3 Every arrow in the diagram

Arrow	From → To	What travels along it
1	Clients → Listener	An encrypted request arrives over the network.

Arrow	From → To	What travels along it
2	Listener → Connection manager	Decrypted bytes are placed into a buffer owned by a worker thread.
3	Connection manager → Filter chain	The request is handed to the ordered list of filters.
4	Filter chain → Rust hook shim	The filter that used to call the C++ parser now calls the shim instead.
5	Shim → Rust entry API	A C-ABI function call carrying a pointer and length to the request bytes plus an opaque context handle. It crosses the FFI zone.
6	Entry API → Parser crate	The validated byte slice is borrowed (not copied) by the parser.
7	Parser crate → Policy engine	A structured, borrowed view of the request (method, path, headers, claims).
8	Policy engine → Verdict enforcement	A small verdict code (allow, deny, rewrite) or error status returns across the FFI to C++.
9	Verdict → Router	Allowed requests continue to routing; denied ones are answered here and stop.
10	Router → Backends	The request is forwarded to a healthy backend server.
F	Shim → Legacy parser → Verdict	Used only when the rollout gate says “not Rust” or when Rust is unavailable.
P1	Control plane → Async runtime	A notification that a new policy bundle version exists; a background task fetches it.
P2	Async runtime → Policy loader	The downloaded bundle bytes are handed over for parsing and validation.
P3	Policy loader → Snapshot store	A fully validated, immutable policy object is published.
P4	Snapshot store → Policy engine	Every request reads the current snapshot (a cheap reference count increase, no lock).
T1	Policy engine → Telemetry	Decision, timing and (in shadow mode) the comparison with C++.
T2	Telemetry → Observability	Aggregated counters and histograms are exported to dashboards and alerts.

4.4 Why the diagram is shaped this way

- **Rust sits beside C++, not in front of it.** Traffic still enters and leaves through C++. Rust is called like a helper. This is what makes “without rewriting the C++ stack” possible.
- **Only two arrows cross the FFI zone on the request path (5 and 8).** Fewer crossings mean less overhead and a smaller surface for mistakes.
- **The policy path (P1–P4) never touches the request path directly.** Slow work happens in the background; requests only read a ready-made snapshot.
- **The legacy path (F) stays drawn with dashes.** It is temporary, used for safety during rollout and removed afterwards.

5. Components Explained in Detail

This section goes through every component from the diagram, end to end. Each component has a quick fact table followed by a step-by-step explanation of what happens inside it.

5.1 External systems

Clients / Internet traffic

What it is	Every sender of requests: web browsers, mobile apps, scripts, partner services, and potentially attackers.
Simple analogy	Visitors arriving at the building's front door.
Why it matters	Their input is untrusted. A malicious client can send deliberately malformed headers, gigantic cookies, broken encodings or forged tokens to trigger parsing bugs.

How it works, step by step

1. The client opens a TCP connection to the proxy's public address.
2. It performs a TLS handshake (agreeing on encryption keys).
3. It sends an HTTP request: method, path, headers and possibly a body.
4. It waits for a response, which may come from a backend (allowed) or directly from the proxy (denied or error).

Control plane / policy source

What it is	A management service that stores security rules and publishes them as versioned bundles.
Simple analogy	Head office that prints and sends out the updated rule book.
Output	A policy bundle (for example JSON or a binary format) plus a version number and a checksum or signature.

How it works, step by step

1. An operator or automated system changes a rule (for example blocks a new IP range).
2. The control plane bundles all rules into one document and gives it a new version number.
3. It signs or checksums the bundle so receivers can detect tampering or corruption.
4. It notifies proxies that a new version is available (push) or proxies poll for it (pull).

5.2 C++ proxy components (existing)

1. Listener + TLS termination

What it is	The network front door of the proxy, written in C++.
Simple analogy	The front door with a security guard who unlocks sealed envelopes.
Input / Output	Input: encrypted bytes from the network. Output: decrypted plain bytes.
Changed?	No. Left exactly as it was.

How it works, step by step

1. Listens on a port (for example 443) for new connections.
2. Accepts the connection and assigns it to a worker thread.
3. Performs the TLS handshake and decrypts incoming data.
4. Hands the plain bytes to the connection manager.

2. Connection manager

What it is	The C++ event loop and worker-thread system that juggles thousands of open connections.
Simple analogy	A team of receptionists, each handling many visitors in turn without waiting idly.
Key detail	It owns the memory buffers holding request bytes. This matters for the FFI: Rust only borrows these bytes and never frees them.
Changed?	No.

How it works, step by step

1. Reads decrypted bytes into a buffer owned by the current worker thread.
2. Detects when a full request header block has arrived.
3. Creates a per-request context object (C++ side).
4. Starts the filter chain for that request.

3. Filter chain

What it is	An ordered pipeline of small processing steps (filters) every request goes through.
Simple analogy	A series of checkpoints in a corridor.
Changed?	Only configuration: the filter that used to call the C++ parser/policy code now calls the Rust hook shim.

How it works, step by step

1. Runs early filters (for example connection limits, basic sanity checks).
2. Reaches the parsing/policy filter position.
3. Calls the Rust hook shim instead of calling the old C++ code directly.
4. Waits for the verdict and continues or stops depending on it.

4. Rust hook shim + rollout gate

What it is	A small new piece of C++ code (a few hundred lines) that wraps every call into Rust.
Simple analogy	A switchboard operator who decides whether to route a call to the new office or the old office.
Responsibilities	Rollout decision, ABI version check result, building the FFI call, interpreting the returned status, falling back when needed.
Concepts used	Feature flags, percentage rollout, stable hashing, RAII guards in C++ to free Rust handles.

How it works, step by step

1. At start-up, loads the Rust library and calls `rs_abi_version()`. If the version is not one the shim supports, Rust is disabled and every request uses the legacy path (and an alert fires).
2. For each request, asks the rollout gate: off, shadow, or on? The gate can be controlled per percentage, per route or per customer, using a stable hash of a request ID so the same client consistently gets the same path.
3. If **on**: builds the call with a pointer and length to the request bytes and the opaque Rust context handle, then calls the Rust entry API (arrow 5).
4. If **shadow**: calls both the legacy parser (whose answer is used) and Rust (whose answer is only recorded and compared).
5. If **off** or Rust returned an internal error: uses the legacy C++ parser (arrow F).
6. Translates the returned status code into the proxy's own verdict type and hands it to verdict enforcement.

5. Legacy C++ parser (fallback)

What it is	The original C++ parsing and policy code.
Simple analogy	The old lock kept in a drawer in case the new lock jams.
Lifecycle	Kept fully working during rollout; removed (or kept only for emergency) after Rust reached 100% and proved stable.

How it works, step by step

1. Receives the request only when the shim chooses the fallback path.
2. Parses and evaluates rules exactly as before the migration.
3. Returns its verdict to verdict enforcement.
4. In shadow mode, its answer is the one actually used, while Rust's answer is compared against it.

6. Verdict enforcement

What it is	The C++ step that acts on the final decision.
Simple analogy	The guard who either opens the gate, sends the visitor away, or stamps a note on their papers.
Possible actions	Allow, deny (send 403/401), reject malformed (400), rate-limit (429), rewrite or add headers, or fail-safe behaviour on internal error.

How it works, step by step

1. Reads the verdict code (from Rust or legacy).
2. If deny or reject: writes the error response directly to the client and stops processing.
3. If rewrite: applies header changes that Rust described (Rust returns instructions; C++ performs the edit in its own buffers).
4. If allow: passes the request on to the router.

7. Router + load balancer

What it is	The C++ component that picks the destination backend.
Simple analogy	The receptionist telling the visitor which floor and which desk to go to.
Changed?	No.

How it works, step by step

1. Matches the request path/host to a backend cluster.

2. Chooses a healthy server using the load-balancing algorithm (for example round robin or least connections).
3. Forwards the request and later streams the response back to the client.

5.3 The FFI boundary (the doorway between C++ and Rust)

The FFI boundary is a contract written as a C header file (often generated automatically from Rust using a tool such as `cbindgen`). It has five rules, each shown as a small box in the diagram.

Rule 1: ABI version handshake

What it is	A function that returns the interface version the Rust library speaks, checked once at start-up.
Simple analogy	Two people checking they both have the same edition of the instruction manual before working together.
Why it is needed	C++ and Rust are built and deployed separately. Without a version check, a mismatched pair could misread each other's data and crash or misbehave silently.

How it works, step by step

1. The shim calls `rs_abi_version()` right after loading the Rust library.
2. Rust returns a number such as `2`, plus optional feature flags (which optional capabilities are present).
3. The shim compares it with the versions it supports (for example 1 and 2).
4. Match: Rust is enabled. Mismatch: Rust is disabled, the legacy path is used, and an alert is raised.
5. When a new version (v3) is introduced, Rust can export v2 and v3 functions side by side, so C++ can upgrade later. This is what makes incremental rollout across many deployments safe.

Rule 2: Opaque handles

What it is	C++ receives Rust objects only as anonymous pointers (`rs_ctx*`) whose internal layout it cannot see.
Simple analogy	A cloakroom ticket: you can hand it back to get your coat, but you cannot open the cloakroom yourself.
Why it is needed	Rust can change its internal structures freely without breaking C++, and C++ can never accidentally corrupt Rust data.

How it works, step by step

1. At start-up, C++ calls `rs_ctx_new()`; Rust creates a context, boxes it, and returns a raw pointer.

2. C++ stores this pointer and passes it back with every call.
3. Rust converts the pointer back into a safe reference, checking for null first.
4. At shutdown, C++ calls `rs_ctx_free()` exactly once (enforced by a C++ RAII wrapper), and Rust drops the object.

Rule 3: ptr + len byte slices

What it is	Data always crosses as a pointer to the first byte plus an explicit length, never as a null-terminated string.
Simple analogy	Giving someone a page number and the number of lines to read, instead of saying “read until you see a blank line”.
Why it is needed	It prevents reading past the end of the buffer (a classic overflow) and lets Rust build a safe `&[u8]` slice that borrows C++ memory without copying it.

How it works, step by step

1. C++ passes `const uint8_t* data, size_t len` pointing into its own request buffer.
2. Rust checks that the pointer is not null and the length is within sane limits.
3. Rust creates a borrowed slice `&[u8]` with a lifetime limited to this single call.
4. Because the borrow ends when the call returns, Rust can never hold on to C++ memory after C++ frees it.

Rule 4: Status codes (no panics cross)

What it is	Every Rust function returns a simple integer status (for example 0 = OK, 1 = malformed input, 2 = denied, 3 = internal error).
Simple analogy	Using traffic-light colours instead of long explanations, so both sides always understand each other.
Why it is needed	A Rust panic (an unexpected crash) unwinding into C++ is undefined behaviour. Converting everything to codes keeps the boundary predictable.

How it works, step by step

1. Inside Rust, all work returns `Result<Verdict, Error>`.
2. The entry API wraps the work in `std::panic::catch_unwind` so even an unexpected panic is caught.
3. `Ok(verdict)` maps to a verdict code; each `Err(kind)` maps to a specific status code; a caught panic maps to “internal error”.
4. C++ handles each code explicitly (for example, internal error triggers fallback to the legacy path).

Rule 5: Clear memory ownership

What it is	A written rule for every piece of memory: who allocates it and who frees it.
Simple analogy	"Whoever opens the bottle closes it."
The rules used	C++ owns request buffers; Rust only borrows them. Rust owns contexts and any result strings it creates; C++ must give them back through a Rust free function. Memory is never freed by the "other" allocator.

How it works, step by step

1. Request bytes: allocated and freed by C++; borrowed by Rust for one call.
2. Context handles: allocated by Rust (`rs_ctx_new`), freed by Rust (`rs_ctx_free`).
3. Small results (verdict, header-rewrite instructions): written by Rust into a fixed-size struct that C++ provides, so no allocation crosses the boundary on the hot path.
4. Any rare larger output allocated by Rust is returned with a matching `rs_buf_free` function.

Illustrative C header for the boundary:

```
/* rs_policy.h  - generated by cbindgen, ABI version 2 */
uint32_t      rs_abi_version(void);
rs_ctx*       rs_ctx_new(const rs_config* cfg, rs_status* out_status);
void          rs_ctx_free(rs_ctx* ctx);
rs_status     rs_eval_v2(const rs_ctx* ctx,
                         const uint8_t* req, size_t req_len,
                         rs_verdict* out_verdict);
```

5.4 Rust components (new)

A. Async runtime + bounded pool

What it is	A Tokio runtime running on a small, dedicated set of background threads, separate from the C++ worker threads.
Simple analogy	A back-office team that handles paperwork so the front-desk staff never have to stop serving visitors.
Concepts used	Async/await, Tokio tasks, `tokio::sync::Semaphore` for bounded concurrency, timeouts, retries with back-off.
Why bounded	Without a limit, a burst of updates or a slow control plane could spawn unlimited tasks, eating memory and CPU that the data plane needs.

How it works, step by step

1. Started once when the Rust context is created, with a fixed small number of threads (for example 1–2).

2. Receives “new policy available” notifications from the control plane (arrow P1).
3. Before starting a fetch, a task must acquire a permit from the semaphore (for example a maximum of 4 concurrent fetches). If none are free, it waits.
4. Each fetch has a timeout; on failure it retries with exponential back-off, and the current policy keeps working meanwhile.
5. Downloaded bytes are passed to the policy loader (arrow P2); the permit is released automatically when the task finishes (thanks to ownership: the permit is dropped).
6. Importantly, the request path never waits on this runtime. Requests are handled synchronously and only read the ready snapshot.

B. Policy loader + validator

What it is	Rust code that turns a raw policy bundle into a validated, typed policy object.
Simple analogy	A clerk who proof-reads the new rule book before it is put on the guard’s desk.
Concepts used	Result-based error handling with the `?` operator, strongly typed structs, enums for rule types, custom error types.

How it works, step by step

1. Verifies the checksum or signature; on mismatch returns `Err(PolicyError::Integrity)`.
2. Checks the version is newer than the current one; older or equal versions are ignored.
3. Parses the bundle into typed structures (for example with `serde`); malformed input returns `Err(PolicyError::Parse)`.
4. Validates meaning: IP ranges well-formed, regexes compile within size limits, no conflicting rules, all referenced lists exist.
5. Pre-compiles fast lookup structures (prefix tries for IPs, compiled regex sets, hash maps for headers).
6. Returns `Ok(Policy)`. Any `Err` keeps the old policy active and emits a metric and log, so a bad update can never take down traffic.

C. Policy snapshot store

What it is	A tiny component holding the current policy as an immutable, shared, reference-counted object (`Arc<Policy>`) behind an atomic pointer (for example the `arc-swap` crate).
Simple analogy	A notice board where the manager replaces the whole sheet in one move. Readers always see either the full old sheet or the full new sheet, never half of each.
Concepts used	Ownership via `Arc` (shared ownership), immutability, atomic swap, lock-free reads, `Send` and `Sync` traits for thread safety.

How it works, step by step

1. The loader publishes a new `Arc<Policy>` with a single atomic store (arrow P3).
2. Each request loads the current pointer and increments a reference count (arrow P4). No lock, so C++ worker threads never block each other.
3. In-flight requests keep using the version they started with; new requests see the new version immediately.
4. When the last request using the old version finishes, its reference count reaches zero and Rust frees it automatically. No use-after-free is possible.

D. Rust entry API (extern "C")

What it is	The thin set of functions exported to C++ with C-compatible signatures. It is the only `unsafe` area of the Rust code, kept small and heavily reviewed.
Simple analogy	The customs desk at a border: every item is checked before it enters the safe country.
Concepts used	`extern "C"`, `#[no_mangle]`, `unsafe` blocks limited to pointer conversion, `catch_unwind`, `#[repr(C)]` structs.

How it works, step by step

1. Receives the call from the shim (arrow 5) with the context handle, request pointer and length, and an output struct pointer.
2. Checks every pointer for null and the length against a maximum allowed size.
3. Converts raw pointers into safe Rust references and a borrowed `&[u8]` slice (the only `unsafe` step, with a documented safety comment).
4. Wraps the rest of the work in `catch_unwind`, then calls the parser (arrow 6).
5. Writes the verdict into the caller-provided output struct and returns a status code (arrow 8).

Illustrative entry function:

```
#[no_mangle]
pub extern "C" fn rs_eval_v2(ctx: *const Ctx, req: *const u8, len: usize,
                             out: *mut RsVerdict) -> RsStatus {
    if ctx.is_null() || req.is_null() || out.is_null() || len > MAX_REQ {
        return RsStatus::BadArgs;
    }
    // SAFETY: C++ guarantees `req` is valid for `len` bytes during this call.
    let bytes: &[u8] = unsafe { std::slice::from_raw_parts(req, len) };
    let ctx = unsafe { &*ctx };
    match std::panic::catch_unwind(|| ctx.evaluate(bytes)) {
        Ok(Ok(v))   => { unsafe { *out = v.into() }; RsStatus::Ok }
        Ok(Err(e))  => e.into(),           // typed error -> status code
        Err(_)      => RsStatus::Internal, // panic caught, never crosses
    }
}
```

E. Parser crate

What it is	A pure Rust library that turns raw request bytes into a structured view: method, path, query, headers, cookies, and token claims (for example from a JWT).
Simple analogy	A careful reader who highlights the important parts of a document without retyping it.
Concepts used	Borrowing and lifetimes (`Request<'a>` holds `&'a [u8]` slices), zero-copy parsing, bounds-checked indexing, `Result` for every failure, strict limits.
Testing	Fuzz testing (cargo-fuzz), property tests, and differential tests that compare its output against the legacy C++ parser on millions of real and generated inputs.

How it works, step by step

1. Receives the borrowed byte slice (arrow 6).
2. Splits the request line and validates method, path and version against allowed characters.
3. Iterates through header lines, enforcing limits (maximum header count, maximum header size, maximum total size).
4. Rejects dangerous ambiguities explicitly: duplicate `Content-Length`, conflicting `Transfer-Encoding`, invalid percent-encoding, bare line feeds. These are classic request-smuggling tricks.
5. Each header name and value is stored as a borrowed slice pointing into the original buffer; nothing is copied.
6. Decodes token parts (for example base64url JWT segments) with checked lengths.
7. Returns `Ok(Request<'a>)` (arrow 7) or `Err(ParseError::...)`, which becomes a “malformed request” status.

F. Policy engine (traits)

What it is	The decision maker. It runs the current set of rules over the parsed request and combines their outcomes into one verdict.
Simple analogy	A panel of specialist inspectors; each gives an opinion, and a clear procedure decides the final answer.
Concepts used	Traits (`Rule`, `Matcher`), trait objects or generics, enums for `Decision`, short-circuit evaluation, `Send + Sync` bounds so rules are thread-safe.

How it works, step by step

1. Loads the current policy snapshot (arrow P4).
2. Iterates through the rules in priority order. Every rule implements the same `Rule` trait, so the engine treats an IP block list, a path rule, a JWT check and a rate-limit rule identically.
3. Each rule returns `Result<Decision, PolicyError>` where `Decision` is `Allow`, `Deny(reason)`, `Rewrite(instructions)` or `Continue`.
4. The engine stops at the first definitive `Deny` (short-circuit) to save time.
5. Combines results into a final verdict and records which rule decided (useful for logs).
6. Sends the verdict back to C++ through the entry API (arrow 8) and reports to telemetry (arrow T1).

Illustrative trait:

```
pub trait Rule: Send + Sync {
    fn name(&self) -> &str;
    fn evaluate(&self, req: &Request<'_>) -> Result<Decision, PolicyError>;
}

pub enum Decision { Allow, Deny(Reason), Rewrite(Vec<HeaderEdit>), Continue }
```

G. Telemetry + shadow compare

What it is	Lightweight metric collection and the comparator used during rollout.
Simple analogy	A scoreboard plus a referee who checks whether the new player makes the same calls as the old one.
Concepts used	Atomic counters, histograms, sampling, a bounded channel so metrics never block requests.

How it works, step by step

1. Counts every outcome (allow, deny, malformed, internal error) and records latency in a histogram (arrow T1).
2. In shadow mode, receives both answers (Rust and legacy) and records whether they match. Mismatches are logged with a sample of the request (redacted) for investigation.

3. Sends data to a bounded queue; if the queue is full, it drops samples instead of slowing down traffic.
4. Periodically exports aggregates to the observability stack (arrow T2).

Upstream backends and observability stack

Upstream backends	The real applications. They receive only requests that passed the policy check, so they are protected by the new safer parsing.
Observability stack	Dashboards (latency, error rates, mismatch rate), alerts, and the evidence used to approve each rollout step.

How it works, step by step

1. Backends process allowed requests and return responses through the C++ proxy.
2. Engineers watch the dashboards; when mismatch rate is zero and latency is unchanged, they widen the rollout.
3. Alerts trigger automatic or manual rollback (switching the gate back to legacy) if something looks wrong.

6. Workflows Explained End to End

A workflow is a sequence of steps over time. This project has several: start-up, the live request, the fallback, policy updates, error handling, telemetry, the rollout itself, and the engineering process used to migrate each component.

6.1 Start-up workflow (when the proxy boots)

1. The C++ proxy process starts and reads its configuration, including the rollout settings.
2. It loads the Rust library (linked statically into the binary, or loaded as a shared library).
3. The shim calls `rs_abi_version()`. If the version is unsupported, Rust is disabled, an alert fires, and the proxy continues with the legacy path. The proxy never fails to start because of Rust.
4. The shim calls `rs_ctx_new(config)`. Rust builds its context: creates the Tokio runtime with a fixed number of background threads, creates the semaphore, and loads the initial policy (from a local file or the control plane).
5. The initial policy goes through the full loader/validator. If it fails, Rust returns an error status and the shim keeps Rust disabled until a valid policy arrives.
6. Rust returns an opaque context handle. The shim stores it inside a C++ RAII wrapper that will call `rs_ctx_free()` exactly once at shutdown.
7. Optional warm-up: a few synthetic requests are evaluated to warm caches before live traffic is accepted.
8. The listener starts accepting connections.

6.2 Live request workflow (the hot path, arrows 1–10)

This is what happens for every single request, typically within microseconds.

Step	Where	What happens, in simple words
1	Client → Listener	The client connects and sends an encrypted request. The listener completes TLS and decrypts the bytes.
2	Listener → Connection manager	The decrypted bytes are stored in a buffer owned by a C++ worker thread. A request context is created.
3	Connection manager → Filter chain	The request starts moving through the ordered filters.
4	Filter chain → Shim	The parsing/policy filter calls the shim. The rollout gate decides: Rust, shadow, or legacy. Assume Rust.
5	Shim → Rust entry API	One C function call: context handle + pointer + length + output struct. No copying, no allocation, no network hop.

Step	Where	What happens, in simple words
5a	Inside entry API	Null checks and size checks. The raw pointer becomes a borrowed `&[u8]` valid only for this call. A panic guard is set up.
6	Entry API → Parser	The parser reads the bytes in place, enforces limits, rejects ambiguous or malformed input, and builds a borrowed `Request` view.
7	Parser → Policy engine	The engine loads the current policy snapshot (lock-free), runs each `Rule` in order, and stops early on a definitive deny.
8	Policy engine → Verdict (C++)	The decision is written into the C++-provided output struct; a status code is returned. The borrow of the request bytes ends here.
8a	Shim	Translates the status code into the proxy's verdict type. Internal error triggers fallback (section 6.3).
9	Verdict → Router	Allow: continue to routing. Deny/malformed: C++ writes an error response (400/401/403/429) and stops. Rewrite: C++ applies the header edits.
10	Router → Backend	The router chooses a healthy backend and forwards the request; the response travels back through the proxy to the client.
T1	Background	Telemetry records the outcome and latency through a non-blocking bounded queue.

> **Why the hot path stays fast**
>
> Only one function call crosses into Rust and one result comes back. The request bytes are borrowed, never copied. No memory is allocated across the boundary. The policy is read without locks. No async runtime is involved per request, so there is no waiting or task switching.

6.3 Fallback workflow (grey dashed arrows F)

The fallback keeps traffic flowing whenever Rust should not or cannot answer.

1. **Trigger conditions:** the rollout gate says "legacy" for this request; the ABI version check failed at start-up; Rust has no valid policy yet; or Rust returned an internal-error status (including a caught panic).
2. The shim calls the legacy C++ parser/policy code exactly as the old system did.
3. The legacy verdict goes to verdict enforcement, and the request continues normally.
4. A metric records that fallback happened and why; repeated internal errors fire an alert.

5. **Important distinction:** a “malformed request” or “denied” answer from Rust is a correct, final decision and does **not** trigger fallback. Only Rust’s own internal problems do. Otherwise attackers could bypass Rust simply by sending bad input.

6.4 Policy update workflow (brown arrows P1–P4)

1. **P1 – notification:** the control plane publishes policy version N+1. The Rust async runtime learns about it (push notification or periodic poll).
2. **Bounded fetch:** a Tokio task acquires a semaphore permit (waiting if the limit is reached), then downloads the bundle with a timeout and retries with back-off on failure.
3. **P2 – hand-off:** the downloaded bytes are passed to the policy loader.
4. **Validate:** integrity check, version check, parsing, semantic validation and pre-compilation. Every step returns `Result`; the first `Err` stops the process.
5. **On failure:** the current policy stays active, an error metric and log are emitted, and the next attempt happens later. Traffic is unaffected.
6. **P3 – publish:** on success, the new immutable `Arc<Policy>` is installed with one atomic swap.
7. **P4 – use:** new requests immediately read version N+1; in-flight requests finish on version N.
8. **Clean-up:** when the last request holding version N finishes, it is freed automatically by Rust’s ownership system.
9. The semaphore permit is released when the task ends, allowing the next queued fetch to proceed.

6.5 Error handling workflow (end to end)

Every failure has a defined path. Nothing is silently ignored and nothing crashes the proxy.

Failure	Where detected	Rust handling	What C++ does	User sees
Malformed request bytes	Parser (E)	`Err(ParseError)` → status MALFORMED	Sends 400 Bad Request	400 error
Rule says deny	Policy engine (F)	`Ok(Deny)` → verdict DENY	Sends 403/401	Access denied
Rate limit exceeded	Policy engine (F)	`Ok(Deny(RateLimited))`	Sends 429	Too many requests
Null pointer / oversize input	Entry API (D)	Returns BAD_ARGS immediately	Logs bug, uses fallback	Normal response
Unexpected panic	Entry API (D)	`catch_unwind` → INTERNAL	Uses legacy fallback, alerts	Normal response
Bad policy bundle	Loader (B)	`Err(PolicyError)`, old policy kept	Nothing (not on hot path)	Nothing

Failure	Where detected	Rust handling	What C++ does	User sees
Control plane down	Async runtime (A)	Timeout, retry with back-off	Nothing	Nothing
ABI mismatch	Shim at start-up	Version reported honestly	Disables Rust, alerts	Nothing

6.6 Telemetry workflow (green arrows T1–T2)

1. After each decision, the engine pushes a tiny event (outcome, rule, latency) into a bounded, non-blocking queue.
2. If the queue is full, the event is dropped and a "dropped" counter increases. Telemetry can never slow requests down.
3. In shadow mode, both the Rust and legacy verdicts are recorded and compared; mismatches are sampled and redacted.
4. A background task aggregates counters and latency histograms and exports them (T2) to the observability stack.
5. Dashboards show: Rust vs legacy latency (p50, p99, p99.9), CPU per request, memory, error rates, mismatch rate, fallback rate.

6.7 Incremental rollout workflow

This is how the switch from C++ to Rust happened safely in production, controlled by the rollout gate in the shim.

Stage	What happens	Moves on when	Rollback
0. Build and test	Rust code merged behind a flag that is off. Unit, fuzz, property and differential tests run in CI. Benchmarks compare with C++.	All tests pass; benchmarks equal or better.	Not deployed to traffic.
1. Shadow mode	Rust runs on a sample of real traffic in parallel. Legacy answer is used; Rust answer is only compared.	Mismatch rate is zero (or every mismatch explained); no crashes; latency overhead acceptable.	Turn sampling off.
2. Canary (about 1%)	Rust's answer is used for a small, stable slice of traffic on a few hosts.	Error rates, latency and CPU match the control group.	Flip gate to legacy, instantly.

Stage	What happens	Moves on when	Rollback
3. Gradual ramp	10% → 25% → 50% of traffic, region by region.	Metrics stay flat at each step for an agreed soak time.	Flip gate to legacy.
4. Full (100%)	All traffic uses Rust; legacy remains available as emergency fallback.	Stable for weeks.	Flip gate to legacy.
5. Clean-up	Legacy code removed or frozen; new ABI versions can now evolve freely.	–	Redeploy previous build.

The **versioned FFI** is what allows this to work across a large fleet: some hosts can run a newer Rust library (speaking v2 and v3) while their C++ shim still uses v2. Both sides upgrade independently without a big-bang deployment.

6.8 Engineering workflow (how each component was migrated)

1. **Pick the target:** rank C++ components by risk (handles untrusted input, history of bugs, complexity) and by how cleanly they can be separated. Parsers and policy evaluation scored highest.
2. **Draw the boundary:** define the smallest possible C interface (few functions, simple types, ptr+len, status codes) and give it a version number.
3. **Write the Rust component:** pure safe Rust for the logic, a tiny audited `unsafe` layer only in the entry API.
4. **Prove equivalence:** differential testing feeds the same inputs (recorded production samples and fuzz-generated inputs) to both the C++ and Rust implementations and compares outputs.
5. **Fuzz:** run cargo-fuzz continuously on the parser and on the FFI entry functions to find edge cases.
6. **Benchmark:** microbenchmarks (criterion) for parsing and evaluation; end-to-end load tests on the proxy measuring latency percentiles and CPU.
7. **Integrate:** add the shim and rollout gate in C++; build Rust into the proxy build (Cargo invoked from the existing build system, header generated by cbindgen).
8. **Roll out:** follow the rollout stages in section 6.7.
9. **Repeat** for the next component, reusing the same boundary patterns.

7. How Performance Stayed Unchanged

"Unchanged data-plane performance" means users did not experience any extra delay and the servers did not need more CPU or memory. These design choices made that possible:

Technique	Simple explanation	Effect
Same process, direct calls	Rust is a library inside the proxy, not a separate service.	A call costs nanoseconds, not a network round trip.
Zero-copy borrowing	The parser points into the C++ buffer instead of copying it.	No extra memory traffic or allocation per request.
One crossing each way	Only arrows 5 and 8 cross the FFI on the hot path.	Minimal boundary overhead.
No allocation across the boundary	Results go into a fixed struct provided by C++.	No allocator contention between languages.
Lock-free policy reads	`Arc` + atomic swap snapshot.	Worker threads never block each other.
Async kept off the hot path	Tokio only runs background policy work.	No task switching or waiting per request.
Bounded background work	Semaphore caps concurrent fetches; small fixed thread count.	Background work cannot steal CPU from traffic.
Precompiled rules	Tries, hash maps and regex sets built at load time.	Per-request evaluation is fast and predictable.
Optimised builds	Release mode, link-time optimisation, same CPU target as C++.	Code speed comparable to C++.
Measured, not assumed	Benchmarks in CI plus canary comparisons in production.	Regressions caught before full rollout.

8. Security Benefits and What the Project Achieved

8.1 Security benefits

- **Whole bug classes removed** in the riskiest code: buffer overflows, use-after-free, double free and data races are prevented by the Rust compiler.
- **Smaller attack surface:** the only `unsafe` Rust code is the thin entry layer, which is small enough to review line by line.
- **Explicit error handling:** `Result` forces every failure to be handled; malformed input becomes a clean 400 response instead of undefined behaviour.
- **Request-smuggling defences:** strict, unambiguous parsing rules reject the tricks attackers use to confuse proxies and backends.
- **Safe policy updates:** a bad or tampered policy is rejected and the previous one keeps working.
- **Contained failures:** panics are caught at the boundary; the proxy falls back instead of crashing.

8.2 Engineering benefits

- **Low risk:** the proven C++ proxy was not rewritten.
- **Reversible:** every rollout step could be undone in seconds.
- **Reusable pattern:** the versioned FFI and shim approach can be repeated for other components.
- **Extensible policies:** new rules are added by implementing the `Rule` trait, without touching the engine.

9. Quick Recap

Question	Short answer
What was the problem?	Security-critical parsing and policy code in a C++ proxy was exposed to memory-safety bugs.
What was done?	Those components were rewritten in Rust; the rest of the C++ proxy was kept.
How do C++ and Rust talk?	Through a versioned C ABI: opaque handles, ptr+len slices, status codes, clear ownership, no panics crossing.
Which Rust features mattered?	Ownership and borrowing (safe zero-copy), traits (pluggable rules), Result (forced error handling), async (background updates), bounded concurrency (overload protection).
How was it switched on?	Shadow mode → canary → gradual ramp → 100%, with instant fallback to legacy C++.
Did it slow things down?	No. Same-process calls, zero copies, lock-free reads and no async on the hot path kept performance unchanged.
What was the outcome?	A safer proxy, same speed, delivered without a risky big-bang rewrite.